Design, Build, Ship

The University of Chicago
Spring 2026 - Week 4

Agenda

🔥 Warmup: Architecture Challenge

💻 Claude Code → Codex

📋 Assignment 3 Review

📖 Reading + Discussion

☕ Break

🔨 Build Together

🚀 Deploy & Go Live

🤝 Share-Outs

📝 Assignment 4

🔥 Warmup: Architecture Challenge

In groups: design the architecture for a live NBA scoreboard.

ARCHITECTURE CHALLENGE

🤔 The First Instinct

Browser → ESPN API → Display scores
Browser → ESPN API → Display scores
Browser → ESPN API → Display scores
× 500 users = 500 calls every few seconds

Why this breaks:

Scale
500 users × every 5 seconds = 6,000 API calls per minute. They'll rate-limit or block you.

CORS
The sports API doesn't send CORS headers — the browser can't call it directly.

Redundancy
The Lakers score is the same for everyone. Why fetch it 500 times?

No Persistence
Refresh the page, scores vanish. No history.

Auth + Favorites
Where do user accounts live? The browser can't store that across devices.

ARCHITECTURE CHALLENGE

✅ The Optimal Architecture

Sports API

↑ polls every 30s

Background Server

↓ writes scores

Database + Auth + Realtime

↓ pushes updates

Frontend (web app)

One server fetches scores for everyone.
The database pushes updates — no polling.
Each user sees only their favorite teams.

The user never talks to the background server.
The user never talks to the sports API.
The frontend just listens.

ARCHITECTURE CHALLENGE

🧠 Why This Matters

Mental Models, Not Memorization

Understand how the pieces fit together — so when something breaks, you know where to look.

Builders Move Through Systems

You can vibe-code a single app. You can't vibe-code a system. When three services need to talk to each other, you need to understand the flow. That understanding is what gives you the confidence to build anything.

💻 Claude Code → Codex

CLAUDE CODE → CODEX

🤖 What is Codex CLI?

OpenAI's terminal-based AI coding agent — the direct analog to Claude Code.

Install

npm i -g @openai/codex

Run

codex

Auto (default) — reads files, edits, and runs commands within the working directory. Asks before touching anything outside that scope or using the network.

Read-only — consultative mode. Browses files but won't make changes or run commands until you approve.

Full Access — works across your machine including network access without asking. Use sparingly.

AGENTS.md vs CLAUDE.md

Same concept — a project context file for agents. CLAUDE.md is for Claude Code. AGENTS.md is the open standard. Have both in your project.

📋 Assignment 3 Review

Open each other's apps at your table. Walk through:

🚀 **The App** — what does it do? What API? Show the sign-up flow.

🥞 **The Full Stack** — where does data live? Hardest integration?

🛠️ **What Broke** — deployment issues? Env vars? RLS? Auth state?

💻 **Claude Workflow** — plan mode? CLAUDE.md updates?

📖 Reading & Discussion

Use the printed discussion handout at your table

High Agency
George Mack
Pre-class reading · highagency.com

Writes and Write-Nots
Paul Graham
In-class reading · paulgraham.com

🍫 Break

BUILD TOGETHER

📈 App → System

Week 1 — Static site
1 HTML file → Vercel

Week 2 — Interactive app
Next.js → Vercel

Week 3 — Full-stack app
Next.js → Vercel + Supabase
2 platforms

Week 4 — Complex System
Next.js → Vercel + Railway + Supabase
3 platforms, 3 services, 1 monorepo architecture

BUILD TOGETHER

🔄 Before & After

Your Week 3 app:

Browser → Vercel → Supabase

A straight line. One request, one response.

What you're building tonight:

ESPN API
↑ polls every 30s
Railway Worker
↓ writes scores
Supabase (Realtime)
↓ pushes updates
Vercel (Next.js)
↓ renders
Browser

Services running independently, data flowing between platforms, things happening without a user triggering them.

BUILD TOGETHER

🏟️ A Real System

Think about what has to happen for millions of people to see the same score update at the same time.

Every user's browser can't be calling the NBA directly. At scale, this kind of problem always converges on the same pattern:

Something fetches the data.
Something stores it.
Something pushes it out.

The specific tools differ — AWS, Kafka, Redis, WebSockets — but the shape is always the same. A background process ingests data, a database holds the current state, and a real-time layer pushes changes to connected clients.

The architecture you're building tonight is the same shape. Smaller scale, same pattern.

BUILD TOGETHER

In Weeks 2-3, MCP gave Claude eyes — it could see your browser, talk to your database.

Tonight, MCP becomes something bigger: Claude can operate across your entire system.

In Weeks 2-3, MCP gave Claude eyes — it could see your browser, talk to your database.

Tonight, MCP becomes something bigger: Claude can operate across your entire system.

Supabase MCP
Create tables, check data, enable Realtime, query your database — all from Claude.

Playwright MCP
Open the frontend, verify scores update, test the full user flow.

Future: Deploy MCP
Check worker health, read logs, redeploy — without leaving the terminal.

MCP scales with your architecture. The more services you have, the more powerful Claude becomes — because it can reach into all of them.

Supabase MCP

🔨 Build Together

Live Sports Dashboard — everyone builds the same system

BUILD TOGETHER

🏗️ Phase 1
Scaffold + Plan

Create the monorepo

```
mkdir nba-scoreboard && cd nba-scoreboard
mkdir -p apps/web apps/worker
```

```
nba-scoreboard/
 CLAUDE.md
 apps/
  web/      ← Next.js frontend (→ Vercel)
  worker/   ← Background service (→ Railway)
```

Scaffold the frontend

```
cd apps/web && npx create-next-app@latest .
```

TypeScript yes, Tailwind yes, App Router yes.

Push to GitHub

```
git init && git add . && git commit
```

Create repo, push.

Write a CLAUDE.md at the root

Describe the full architecture: frontend on Vercel, worker on Railway, Supabase for database + auth + Realtime.

Checkpoint

BUILD TOGETHER

🗄️ Phase 2
Supabase + Auth + Realtime

Create tables via Supabase MCP

games table: id, home_team, away_team, scores, status, game_clock, logos, updated_at

favorites table: user_id, team_abbr (unique per user)

Enable Realtime

Supabase dashboard → Database → Replication → Enable on games table.

Enable Auth

Set up RLS policies:

games — readable by everyone

favorites — users can only read/write their own

Add keys to .env.local

NEXT_PUBLIC_SUPABASE_URL

NEXT_PUBLIC_SUPABASE_ANON_KEY

Checkpoint

Tables created, Realtime enabled, auth ready.

BUILD TOGETHER

👤 Phase 3
Auth + Favorites UI

Auth pages

Sign up, log in, sign out. Show user email when logged in.

Team picker /favorites

Toggle favorite on/off — insert/delete from favorites table.

Gold star on selected teams.

"Pick at least 3 teams to follow."

Navigation

Home (scoreboard), My Teams (favorites), Sign In/Out.

Checkpoint

Users can sign up, pick favorite teams, data saves to Supabase.

BUILD TOGETHER

📊 Phase 4
Scoreboard + Realtime

Scoreboard page

Logged in: show games for favorite teams only.

Not logged in: show all games.

Three sections: Live, Upcoming, Completed.

Game cards

Team logos, names, scores.

Live: red dot + game clock ("Q4 2:31")

Upcoming: tip-off time ("9:30 PM ET")

Completed: "Final" badge

Subscribe to Realtime

```
supabase.channel('games')
  .on('postgres_changes', ...)
  .subscribe()
```

When a score changes, the database pushes the update. **No polling. No refresh.**

Checkpoint

Scoreboard renders, updates in real time.

BUILD TOGETHER

⚙️ Phase 5
Background Worker

Set up the worker

cd apps/worker && npm init -y

npm install @supabase/supabase-js

Create index.js

Fetch NBA scoreboard from ESPN API.

Parse: game ID, teams, scores, status, clock, logos.

Upsert into Supabase every 30 seconds.

Environment variables

SUPABASE_URL + **SUPABASE_SERVICE_ROLE_KEY**

Service role key — bypasses RLS. This is trusted infrastructure, not a user.

Test locally

Run the worker. Watch Supabase dashboard update. Watch the frontend update in real time.

Checkpoint

Worker running, scores flowing, frontend updating.

BUILD TOGETHER

🚀 Phase 6
Deploy the System

Deploy the worker to Railway

Sign up at **railway.app** (no credit card needed).

Create a new project. Connect your GitHub repo — Railway auto-deploys on push.

Point Railway to **apps/worker** as the root directory.

Add env vars: **SUPABASE_URL**, **SUPABASE_SERVICE_ROLE_KEY**.

Connect Railway MCP

Claude can now deploy, check status, and view logs directly.

Deploy the frontend to Vercel

Push to GitHub. Import in Vercel.

Add env vars in Vercel dashboard.

Verify end-to-end

Worker on Railway → scores in Supabase → Realtime push → frontend on Vercel.

Share your URL in Slack. 🎉

Checkpoint

Three services, three platforms, one system.

🚀 Share-Outs

Drop your Vercel URL in Slack

🔄 Review:
From Apps to Systems

📋 Assignment 4

Design, Build, Ship

The University of Chicago
Spring 2026 - Week 4